

Person A — Data & Backend Engineer

Codebase PRD — Patient Risk Intelligence MCP Platform

Sprint: June 25 — July 9, 2026 (2 people, 2 weeks)

Companion to: Person B PRD · Combined Sprint PRD · Full Patient Risk Intelligence PRD v1.0

1. Your Mission

You own everything data touches: the 4 MCP servers, both connector types (SQL and vector), the underlying databases, the synthetic data pipeline, and Layer-2 authorization/hardening inside each server. If your servers don't exist, there is nothing for Person B's agent to call — your work is the foundation the entire demo stands on.

Your servers are independently testable from day one — you do not need to wait for Person B's agent, gateway, or identity provider to verify your work. Use curl, MCP Inspector, or pytest directly against each server as you build it.

2. Scope — What You're Building

- 4 MCP servers: vitals_trends, labs_diagnoses, medications_interactions, clinical_notes_search — one shared hardened template.
- Pluggable Connector ABC + 2 implementations: SQL (TimescaleDB/Postgres) and Vector (Qdrant).
- 3 persistent data stores: TimescaleDB (vitals), Postgres (labs_diagnoses + medications_interactions), Qdrant (clinical_notes).
- Synthea-based data pipeline: generate synthetic FHIR patients, load/embed them into the 3 stores above.
- Layer-2 AuthZ inside every server: JWT re-verification, group/scope intersection per tool, explain-denial errors.
- Hardening: audit logging (+purpose_of_access, validated against a fixed enum), egress/SSRF guard, 30s TTL cache, tool-poisoning guard, usage/cost log.
- Self-healing: tenacity retries + Docker restart policies; one rehearsed chaos demo (kill a DB mid-call).
- RBAC automated test suite: 3 roles × ~13 tools, all combinations.
- Synthea data pipeline determinism: fixed seed + a demo-patient alias file so the same patient ID is reproducible across reseeds.
- Clinical note typing: note_type and author_role fields, plus a get_notes_by_type tool — closes a gap where “doctor notes” vs. other note types didn't exist as a queryable distinction.
- docker-compose.data.yml — your half of the infrastructure file (see Section 7).

3. Out of Scope for This Sprint — Not Your Job

These exist in the full platform vision but are deliberately cut for this 2-week sprint. Don't spend time on them.

Item	Why cut / what replaces it
Unity Catalog OSS governance layer	Your Layer-2 RBAC already enforces access on labs_diagnoses without it.
MCP protocol conformance test suite	MCP Inspector manual pass is enough for this sprint.
Full 13-stage CI/CD pipeline	Person B sets up build+test on push only — not your concern.

Auto-generated 20-item documentation	This PRD is the documentation for this sprint.
--------------------------------------	--

4. Your Day-by-Day Schedule

Date	Your task
Thu Jun 25	TimescaleDB + Postgres containers up, schemas created. Start connector_base.py ABC.
Fri Jun 26	Run Synthea, build loader script, populate vitals/labs/meds tables. Hand Person B a STUB server (see Section 6).
Mon Jun 29	Build vitals_trends server: get_vitals_trend, compute_news2_score, list_abnormal_vitals.
Tue Jun 30	Build labs_diagnoses server: get_lab_trend, get_active_diagnoses, get_diagnosis_history.
Wed Jul 1	Build medications_interactions server: get_active_medications, check_drug_interactions, get_polypharmacy_risk.
Thu Jul 2	Add RBAC, audit (+purpose_of_access), egress guard, cache to all 3 servers built so far.
Fri Jul 3	RBAC pytest matrix (3 roles × 9 tools). CHECKPOINT — flag to the team if behind.
Mon Jul 6	Qdrant up. Build vector_connector.py and clinical_notes_search server (semantic_search_notes, get_recent_notes, get_notes_by_type with note_type/author_role fields). Pin the embedding model used here — it must match the loader exactly.
Tue Jul 7	Self-healing on all 4 servers. Rehearse the chaos demo (kill a DB mid-call).
Wed Jul 8	MCP Inspector pass on all 4 servers. Re-verify full RBAC matrix (4 servers × 3 roles, ~13 tools).
Thu Jul 9	Final fixes. Live demo support.

If Friday Jul 3 finds you behind: drop clinical_notes_search and ship a solid 3-source demo. Tell Person B immediately — their agent integration plan depends on knowing this early, not on July 8.

5. Components You Own — Full Detail

5.1 The 4 MCP Servers

Server	Storage	Tools	Scope
vitals_trends	TimescaleDB	get_vitals_trend, compute_news2_score, list_abnormal_vitals	mcp.vitals.read
labs_diagnoses	Postgres	get_lab_trend, get_active_diagnoses, get_diagnosis_history	mcp.labs.read

medications_interactions	Postgres	get_active_medications, check_drug_interactions, get_polypharmacy_risk	mcp.meds.read
clinical_notes_search	Qdrant	semantic_search_notes, get_recent_notes, get_notes_by_type	mcp.notes.read

compute_news2_score implements NEWS2 (published NHS algorithm).
 check_drug_interactions uses a small curated RxNorm rule set — explicitly not a licensed clinical drug database. All outputs are FHIR R4 shaped (Observation, MedicationStatement, Condition, DocumentReference).

5.1.1 Clinical Note Typing (gap fix)

Every Qdrant point's metadata now carries note_type (physician_note | nursing_note | discharge_summary | consult_note) and author_role (physician | nurse | case_manager) — closing a gap where note type and author role were conflated. Synthea generates one note per encounter, written from the encounter provider's perspective; the loader tags these as physician_note/physician by default. Any nursing_note or discharge_summary variants you generate for demo variety must be clearly marked as augmented, not Synthea-native. get_notes_by_type(patient_id, note_type) is the new direct-filter tool, separate from semantic_search_notes's similarity search.

5.1.2 Embedding Model (gap fix)

Pin sentence-transformers/all-MiniLM-L6-v2 (free, open) in both the loader (infra/synthea/load_patients.py) and vector_connector.py's query() method. These MUST use the identical model and version — a mismatch doesn't error, it just silently returns meaningless similarity results, which is a much worse failure mode to debug.

5.1.3 Synthea Determinism (gap fix)

Set SYNTHEA_SEED to a fixed value so the same patient set generates on every reseed. After generation, write infra/synthea/demo_patient_aliases.json mapping friendly demo IDs (e.g. demo-patient-1) to the actual generated patient UUIDs, so the demo script can reliably reference “the same patient” across runs.

5.2 Connector Layer

One Python ABC (connect/auth/schema/query), two implementations: sql_connector.py (TimescaleDB/Postgres, used by 3 servers) and vector_connector.py (Qdrant, used by clinical_notes_search). This is the architecture's core 'pluggable' proof — keep both implementations behind the exact same interface.

5.3 Hardening Modules (backend/shared/)

- auth.py — JWT decode + RBAC engine (Layer 2): groups $\cap$ allowed_groups, scope check per tool, explain-denial 403.
- audit.py — logs every call: who, what, when, outcome, reason, purpose_of_access, trace_id. purpose_of_access is now a fixed enum (gap fix) — deterioration_review, medication_reconciliation, discharge_planning, care_coordination, routine_review — validated, not free text, so audit analytics and the anomaly panel can group meaningfully.

- `egress_guard.py` — each server's connector is hardcoded to its own backend at startup; cannot be redirected at runtime.
- `cache.py` — 30s TTL cache on read-heavy tools (`get_vitals_trend`, `get_lab_trend`).
- `tool_trust.py` — ensures the server only responds through its registered Kong route.
- `usage_log.py` — query count + denial count per role, per week.
- `telemetry.py` — OpenTelemetry setup; trace ID must propagate through every tool call (Person B's agent originates the trace, you continue it).

5.4 Self-Healing

tenacity retries on transient connector failures; Docker restart policies (unless-stopped) on every server and database container. Rehearse one live failure: kill a database container mid-call, show the connector retry and the container restart automatically.

5.5 RBAC Test Suite

`backend/tests/test_rbac_matrix.py` — every (role, tool) combination, asserting allow or deny matches the matrix below exactly:

Role	vitals_trends	labs_diagnoses	medications_interactions	clinical_notes_search
clinical-viewer	Allow	Allow	Deny	Deny
physician	Allow	Allow	Allow	Allow
case-manager	Deny	Deny	Deny	Allow

6. The Contract You Must Honor

Person B's agent and gateway are built against assumptions about your servers. These are fixed — do not change them without telling Person B the same day.

6.1 Day-1 Stub Server (your most time-critical deliverable)

By end of day Fri Jun 26, stand up at least one server — any domain — that returns hardcoded, FHIR-shaped JSON on its real tool names and real Kong route. Person B's Runtime Agent work is blocked without this. Fake data, real shape — this matters more on day 2 than a fully correct vitals_trends server does.

6.2 Token shape you must read (Person B issues this)

```
JWT claims: { "sub": "<user id>", "oid": "<object id, used in audit 'who'>",
  "groups": ["grp-clinical-viewer" | "grp-physician" | "grp-case-manager"],
  "scp": "mcp.vitals.read mcp.labs.read ..." } (space-separated scopes)
```

6.3 Response envelope you must write

Success: FHIR R4 resource JSON (Observation / MedicationStatement / Condition / DocumentReference).

Denial: 403 with { "error": { "code": "forbidden", "reason": "missing scope mcp.meds.read" } } — Person B's chat UI renders this reason directly to the user.

6.4 What you tell Person B, and when

- Day 2 (Jun 26): your stub server's URL and tool names.
- Day 7 (Jul 3) checkpoint: whether you're on track for all 4 servers or dropping clinical_notes_search.
- The moment any tool name, scope string, or response shape changes — same day, not at integration time.

7. Your docker-compose.data.yml

You own this file; Person B owns docker-compose.platform.yml. Merge into one file together on Jul 8.

- timescaledb-vitals, postgres-clinical, qdrant — the 3 data stores
- synthea-loader — one-shot job, restart: "no"
- pgadmin — optional, for browsing your own data while building
- The 4 MCP server containers themselves, once built

8. Your Directory Structure

```

backend/
├── shared/
│   ├── connector_base.py
│   ├── auth.py
│   ├── audit.py
│   ├── telemetry.py
│   ├── tool_trust.py
│   ├── usage_log.py
│   ├── egress_guard.py
│   └── cache.py
├── connectors/
│   ├── sql_connector.py
│   └── vector_connector.py
├── servers/
│   ├── vitals_trends/ (main.py, news2.py, tools.py, blueprint.yaml, Dockerfile)
│   ├── labs_diagnoses/ (same structure)
│   ├── medications_interactions/ (+ interactions.py)
│   └── clinical_notes_search/ (vector connector, not SQL)
├── infra/synthea/
│   └── demo_patient_aliases.json (NEW – friendly ID mapping)
└── tests/
    ├── test_rbac_matrix.py
    └── test_self_healing.py
  
```

9. Tech Stack You'll Use

Component	Choice	Version
Language	Python	3.12 or 3.13
Web framework	FastAPI	0.136.x
MCP SDK	mcp	v1.28.x — pin mcp>=1.27,<2
MCP protocol	Model Context Protocol	2025-11-25
Time-series DB	TimescaleDB	Postgres extension
Relational DB	PostgreSQL	16
Vector DB	Qdrant	built-in dashboard at :6333/dashboard
Synthetic data	Synthea	generates FHIR R4
Resilience	tenacity	retry logic

10. Acceptance Criteria — Definition of Done

- All 4 servers respond correctly via MCP Inspector with real (not stub) data.
- RBAC pytest suite passes all 12 role/tool combinations.
- A 403 denial includes the correct missing-scope reason.
- Killing a database container mid-call triggers visible retry + container auto-restart.
- A repeated identical call to `get_vitals_trend` within 30s is visibly faster (cache hit).
- Every call — allowed or denied — produces a complete audit record including a valid (enum-checked) `purpose_of_access`.
- `get_notes_by_type(patient_id, 'physician_note')` returns only physician-authored notes, correctly distinct from `nursing_note/discharge_summary`.
- Re-running the Synthea loader with the same `SYNTHEA_SEED` produces the same demo patient at the alias in `demo_patient_aliases.json`.
- A semantic search query returns sensible, relevant results — confirming the loader and query embedding models actually match.

11. Daily Sync — What to Report

15 minutes, every day, with Person B. Report: what you finished yesterday, what's blocking you, and any change to the contract in Section 6. Don't wait for the Jul 3 checkpoint to surface a problem you already know about today.